

BUPT SE ICP Quiz -- Unit 08 Pointers

If you are using LLM to help you in this quiz, you MUST write down 1) the name of the LLM and 2) the PROMPTS. Otherwise, you will get zero score if the instructor finds that you are using LLM.
You may use the following table as a template, putting the table down to the title of each question.

Name of LLM	Prompts

- To declare a variable named `flagp` as a pointer to a Boolean value, you write `bool *flagp;_`;
- With the statement `double *p1, p2;` the type of `p1` is `pointer` and the type of `p2` is `integer`_____
- [`T`] True or false: For any variable `x`, the expression `&x` is essentially a synonym for `x`.
- [`F`] True or false: For any variable `x`, the expression `&*x` is essentially a synonym for `x`.
- [`F`] True or false: If `p` is a pointer variable, the expression `p++` adds 1 to the internal representation of `p`.
- The internal representation of the constant `NULL` is `(void*) 0`_____
- Assume that variables of type `double` take up eight bytes. If the base address of the array `doubleArray` is 1000, the address value of `doubleArray + 5` is `1040`_____
- [`F`] True or false: Because C treats arrays as pointers to their initial element, declaring an array variable is identical internally to declaring the same variable as a pointer.
- [`T`] True or false: The operator `++` is applied first in the expression `*p++`.
- Write a statement that allocate a dynamic array called `flags` consisting of 100 Boolean values
`bool *flags = malloc(sizeof *(100* sizeof(bool)));`_____
- Explain the difference between pointer assignment and value assignment.

Name of LLM	Prompts

- Assuming that variables of type `double` require eight bytes of memory and that a pointer requires four, draw a memory diagram showing a portion of the stack frame that contains the following declarations:

`double d1 = 1.0, d2 = 2.0;`

`double *dp1, *dp2;`

Trace through the operation of these statements on the left and give the values to the expression on the right:

<code>dp1 = &d1;</code>	<code>*dp1:</code>
<code>dp2 = &d2;</code>	<code>*dp2:</code>
<code>*dp1 = 3.14159;</code>	<code>d1:</code>
<code>d2 = 2.71828;</code>	<code>*dp2:</code>
<code>dp1 = dp2;</code>	<code>*dp1:</code>

- Write a function `void findMaxMin(int x, int y, int z, int* max, int* min);` which finds the maximum and minimum from `x`, `y` and `z`. The function return the two values by its parameters `max` and `min`. Test the function in `main()`. [利用函数参数传出多个“返回值”]

`#include <stdio.h>`

```
void findMaxMin(int *x, int *y, int *z, int *max, int *min){
    *max = (*x > *y) ? ((*x > *z) ? *x : *z) : ((*y > *z) ? *y : *z);
    *min = (*x < *y) ? ((*x < *z) ? *x : *z) : ((*y < *z) ? *y : *z);
}

int main(){
    int x = 5;
    int y = 10;
    int z = 2;
    int max, min;
    findMaxMin(&x, &y, &z, &max, &min);
    printf("The max is %d, the min is %d\n", max, min);
    return 0;
}
```

14. Write a function `search()` that takes an integer array `arr`, the size of the array `size` and an integer as input, and returns whether `x` exists in `arr`. Test the function in `main()`. [数组作为函数参数会退化为指针，丢失数组大小信息]

```
#include <stdio.h>
#include <stdbool.h>

bool search(int *arr, int size, int x){
    for(int i = 0; i < size; i ++){
        if (arr[i] == x){
            return true;
        }
    }
    return false;
}

int main(){
    const int size = 10;
    int ray[size] = {23,323,13,324,432,5,524,423,43,21};
    if(search(ray, size, 43)){
        printf("yes");
    }
}
```

15. Write a function `compare()` that takes two integer arrays `arr1` and `arr2` as input, and compares whether the two arrays are equal.

```
#include <stdio.h>
#include <stdbool.h>
```

```
bool intCmp(int* str1, int* str2){
    while (*str1 != '\0' && *str1++ == *str2++);
    if (*str1 != *str2) return false;
    return true;
}

int main(){
    int str1[7] = {1, 2, 3, 4, 5, 6, 7};
    int str2[7] = {1, 2, 3, 4, 5, 6, 7};
    if (intCmp(str1, str2)) printf("Equal!");
}
```

16. Write a function `stringLen()` that takes a string `str` as input, and returns the length of `str`.

```
#include <stdio.h>

int stringLen(char *str){
    int n = 0;
    while (*str++ != '\0') n ++;
    return n;
}

int main(){
    char str[] = "HelloWorld";
    printf("strlen is %d", stringLen(str));
}
```

17. Write a program that takes an integer `n` as input from the keyboard, then dynamically allocates memory for `n` integers, reads `n` integers from the keyboard, and finally frees the memory.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    int *arr;

    printf("Enter the number of integers (n): ");
    scanf("%d", &n);

    arr = (int*)malloc(n * sizeof(int));

    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++) {
        printf("Enter integer %d: ", i + 1);
        scanf("%d", &arr[i]);
    }
}
```

```
}

printf("\nYou entered: ");
for (int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

free(arr);
printf("Memory freed successfully.\n");

return 0;
}
```

18. Write a function `char* concatenateStrings(char* s1, char* s2)`; which concatenate two strings. The function will allocate memory to store the concatenated strings.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char* concatenateStrings(char* s1, char* s2){
    char* result = malloc(strlen(s1) + strlen(s2) + 1);
    char* res = result;
    while (*s1 != '\0') *result++ = *s1++;
    while (*s2 != '\0') *result++ = *s2++;
    *(result) = '\0';
    return res;
}

int main(){
    char* s1 = "Hello";
    char* s2 = "World";
    char* result = concatenateStrings(s1, s2);
    printf("The concatenated string is %s", result);
    return 0;
}
```

19. Write a function `int wordCount(char* s)` to count the number of words in a string. Assuming the words are delimited by one or more space character.

```
#include <stdio.h>

int wordCount(char* s){
    int counter = 0;
    while(*s){
```

```
    if(s[0] != ' ') {
        counter ++;
        s++;
    }
    else if(*s == ' ') {
        s++;
        if(*s != ' ')counter++;
    }
    else s++;
}
return counter;
}

int main(){
    char *str = "Hi im xinjian Nice 2 meet u";
    printf("There're %d words in this sentence\n", wordCount(str));
    return 0;
}
```

```
SCAN the string {
    SKIP any spaces
    WordCount++
    FORWARD to next space (SKIP any character that is not space or '\0')
}
```

20. Student Score Statistics. Define a struct Student that includes name (string) and score (integer). Write a function averageScore() which receives an array of Student type and its length through pointers, calculates and returns the average score of all students.

```
#include <stdio.h>

typedef struct{
    char* name;
    int score;
} Student;

float averageScore(Student *student, int lenth){
    float total = 0;
    for (int i = 0; i ≤ lenth; i++){
        total += student[i].score;
    }
    float average = (float)(total/lenth);
    return average;
}
```

21. Write a program that reads simple data declarations and responds with the amount of memory that would be allocated to that variable. For example, the following sample run shows the output of the program for two possible input lines:

Each input line should consist of:

- 1) A type name, which must be one of the following: char, int, short, long, float, or double. (You can obtain the sizes of these types by using the sizeof operator.)
- 2) One or more individual declaration specifications (as outlined below) separated by commas.
- 3) A semicolon marking the end of the line.

Your program should exit if it reads a blank input line.

The individual declaration specifications must consist of a variable name, which can be modified in either or both of the following ways:

- 1) Preceded by an asterisk, to indicate a pointer variable
- 2) Followed by an integer enclosed in brackets, to indicate an array

```
#include <stdio.h>
#include <string.h>
```

```
// 通过 compare 得出对应类型
size_t calcuSize(const char *type) {
    if (strcmp(type, "char") == 0) return sizeof(char);
    if (strcmp(type, "int") == 0) return sizeof(int);
    if (strcmp(type, "short") == 0) return sizeof(short);
    if (strcmp(type, "long") == 0) return sizeof(long);
    if (strcmp(type, "float") == 0) return sizeof(float);
    if (strcmp(type, "double") == 0) return sizeof(double);
    return -1;
}

// Get the variable's name to output
void getName(const char *decl, char *name) {
    const char *start = decl;
    // 跳过空格和 *
    while (*start == ' ' || *start == '*') start++;
    const char *end = start;
    // 找到第一个 [ 或 , 或 ; 或 \0
    while (*end && *end != '[' && *end != ',' && *end != ';' && *end != '\0')
        end++;
    int len = end - start;
    if (len > 0 && len < 100) {
        strncpy(name, start, len);
        name[len] = '\0';
    } else {
        strcpy(name, "var");
    }
}
```

// 计算单个声明的大小

```
size_t getSize(const char *decl, size_t base_size) {
    int is_ptr = 0;
    const char *p = decl;
    while (*p == ' ') p++;
    if (*p == '*') {
        is_ptr = 1;
        while (*p == ' ' || *p == '*') p++; // 支持多个 * (但题目只说一个)
    }

    const char *bracket = strchr(p, '[');
    if (bracket) {
        int n = 1;
        if (sscanf(bracket, "[%d]", &n) == 1 && n > 0) {
            if (is_ptr) return sizeof(void*) * n;
            else return base_size * n;
        }
    }

    return is_ptr ? sizeof(void*) : base_size;
}
```

// 处理一个类型的一行声明

```
void process_line(const char *type, const char *rest) {
    size_t base = calcuSize(type);
    char temp[300];
    strcpy(temp, rest);

    // 去掉分号
    char *semi = strchr(temp, ';');
    if (semi) *semi = '\0';

    char *token = strtok(temp, ",");
    while (token) {
        // 去前后空格
        while (*token == ' ') token++;
        char *end = token + strlen(token) - 1;
        while (end > token && *end == ' ') *end-- = '\0';

        char name[100];
        getName(token, name);
        size_t sz = getSize(token, base);
        printf("%s requires %zu bytes\n", name, sz);
    }
}
```

```
        token = strtok(NULL, ",");
    }
}

// 主函数
int main() {
    char line[500];
    char type[50];

    printf("Enter variable declarations, ending with a blank line.
\n");

    while (1) {
        // 用 scanf 读一整行（跳过前导空白，包括换行）
        if (scanf(" %[^\\n]", line) != 1) {
            // 读到空行或 EOF
            break;
        }

        // 尝试解析类型
        if (sscanf(line, "%49s", type) != 1) break;

        // 找到类型后的剩余部分
        char *rest = strchr(line, ' ');
        if (!rest) continue;
        rest++; // 跳过空格

        process_line(type, rest);
    }

    return 0;
}
```

Enter variable declarations, ending with a blank line.

int x, y;8

x requires 2 bytes

y requires 2 bytes

char c, *cptr, carray[80];8

c requires 1 bytes

cptr requires 4 bytes

carray requires 80 bytes